Repaso Parcial 2 Bases de Datos: ejercicios modelo resueltos

Selección de ejercicios de parciales viejos (2023 a 2025) que resolvimos juntos, ordenados por tema. Cada uno trae el enunciado resumido, la resolución y qué recordar.

Aviso sobre los datos supuestos: en varios enunciados pegados faltaban los tamaños de atributos. Donde supuse algo (4 bytes por atributo, 8 bytes de puntero, 32 bytes para un nombre) lo marco con **[supuesto]**. En el examen, usá lo que diga tu hoja y escribí tu suposición si falta un dato.

---

0. Fórmulas que más se usan

Qué	Fórmula
Tamaño de registro	suma de atributos + 8 (puntero)
Tuplas por bloque	⌊B / tamaño_tupla⌋ (hacia abajo)
Bloques de una tabla	⌈N / tuplas_por_bloque⌉ (hacia arriba)
Claves por nodo B+	n = (C − 8) / (TCB + 8), hacia abajo
Altura B+	h ⩽ ⌈log base ⌈n/2⌉ de K⌉, con K = valores **distintos** de la clave
fs de A = c	1 / V(A, r)
fs de A > c (la filmina usa c+1)	(max − (c+1)) / (max − min)
fs de reunión	1 / max(V(A,r), V(B,s)); con clave foránea: 1/|r|
Loop anidado por bloques	b_r · b_s + b_r (chica de externa)
Loop con M buffers	⌈b_r/(M−2)⌉ · b_s + b_r
Loop indexado	b_r + n_r · c
Merge join	b_r + b_s (más ordenar si hace falta)
Ordenar	b · (2k + 1), k = pasadas de mezcla

Qué	Fórmula
Índice primario, igualdad, clave candidata	h + 1
Índice primario, igualdad, no clave	h + b
Materialización	costos de operadores + escritura de cada intermedio (el resultado final no se escribe)

---

1. Tamaño de resultados intermedios

Ejercicio 1.1: pizzería (parcial 07/11/2024)

π cnombre (cliente ⋈ σ paño=2015 (pedido)). pedido tiene 1.200.000 tuplas, V(pedido, paño) = 40, bloque de 4096 bytes. pedido tiene 7 atributos (cid, zid, phora, paño, pmes, pdia, cantidad).

1. **Probabilidad:** 1/40 = 0,025.
2. **Tuplas de la selección:** 1.200.000 / 40 = **30.000**.
3. **Bloques:** tupla completa de pedido = 7 · 4 + 8 = 36 bytes **[supuesto]** → ⌊4096/36⌋ = 113 por bloque → ⌈30.000/113⌉ = **266 bloques**.

Variante con paño ≥ 2015 (el último año es 2024, 40 años: de 1985 a 2024): fs = (2024 − 2015)/(2024 − 1985) = 9/39 → ≈ 276.923 tuplas → ⌈276.923/113⌉ = **2451 bloques**. Si tu cátedra cuenta 10 años de 40, fs = 1/4, da 300.000 tuplas y 2655 bloques. Usá el criterio visto en clase.

Qué recordar

- La selección deja **tuplas completas** de su tabla. Los bloques se calculan con el tamaño de tupla de **pedido**, no con el de la proyección final ni sumando atributos de cliente (cliente todavía no participó).
- El "último año es 2024" solo importa si la condición es un rango.
- El +8 del puntero es convención de la cátedra para archivos secuenciales. Si el enunciado da el tamaño de tupla, usá ese número directamente.

Ejercicio 1.2: reunión con clave foránea

estudiante (10.000 tuplas) e inscripción (40.000), id_estudiante clave foránea.

fs = 1/10.000 → |resultado| = 10.000 · 40.000 / 10.000 = **40.000 tuplas**.

---

2. Árboles B+: claves por nodo y altura

Ejercicio 2.1: altura (recuperatorio 18/11/2025)

Índice primario sobre id_estudiante de inscripción. 40.000 tuplas, V(id_estudiante) = 9000, nodos de 314 claves.

- K = **9000** (valores distintos de la clave, no las 40.000 tuplas).
- Base ⌈314/2⌉ = 157.
- h = $\lceil \log_{157}(9000) \rceil$ = ⌈ln 9000 / ln 157⌉ = ⌈9,105 / 5,056⌉ = **2**.

Qué recordar

- "Índice primario" significa que el archivo está **ordenado** por ese atributo, no que sea la clave primaria de la tabla.
- V(id_curso) = 450 es un dato distractor: el índice es sobre id_estudiante.

Ejercicio 2.2: claves por nodo (parcial 06/11/2025, ej. 1.3)

Índice sobre año de pedido, bloque de 4096 bytes. Año ocupa 4 bytes **[supuesto]**, puntero 8 bytes.

Un nodo con n claves tiene n + 1 punteros:

- 4n + 8(n + 1) ⩽ 4096 → 12n ⩽ 4088 → n ⩽ 340,67 → **n = 340**.

Es lo mismo que n = (C − 8)/(TCB + 8). El −8 es el puntero extra; TCB + 8 es lo que ocupa cada par clave + puntero.

Ejercicio 2.3: nombres de usuarios (parcial 2023, ej. 1a)

usuario con 100.000 tuplas, índice primario en nombre, 10.000 nombres distintos, nombre no se repite más de 15 veces, bloque de 4096.

- **n:** nombre de 32 bytes **[supuesto]** → (4096 − 8)/(32 + 8) = 102,2 → **n = 102**.
- **Altura:** K = 10.000, base ⌈102/2⌉ = 51 → $\lceil \log_{51}(10.000) \rceil$ = ⌈2,34⌉ = **3**.
- **Operador de la selección:** índice primario, igualdad, clave no candidata: costo **h + b**. Los 15 nombres iguales están contiguos y caben en 1 o 2 bloques → 3 + 2 = **5 transferencias** (4 si caben en uno).

---

3. Elección del algoritmo de reunión

Ejercicio 3.1: tres reuniones comparadas (recuperatorio 18/11/2025)

π nombre (σ nota>8 (estudiante ⋈ inscripción)). estudiante: 10.000 tuplas, 56 por bloque → **179 bloques** (⌈10.000/56⌉). Índice primario sobre id_estudiante en inscripción, un estudiante cursa como máximo 20 cursos.

Loop indexado (externa estudiante, interna inscripción): c = h + b = 2 + 20 = 22. Costo = b_r + n_r · c = 179 + 10.000 · 22 = **220.179**.

Para comparar supuse que inscripción tiene 56 tuplas por bloque **[supuesto]** → 715 bloques:

Algoritmo	Costo
Indexado	220.179
Loop por bloques (estudiante externa)	179 · 715 + 179 = **128.184**
Merge (hay que ordenar estudiante, M = 10 **[supuesto]**)	895 + 894 ≈ **1.789**

Qué recordar

- El indexado solo conviene si n_r · c < b_r · b_s. Acá no conviene (220.000 contra 128.005).
- estudiante está ordenada por **nombre**, no por id: para el merge hay que ordenarla primero.

Ejercicio 3.2: qué algoritmo con cada tipo de índice (parcial 26/10/2023, ej. 3)

ventas ⋈ vendedores por venta_id.

- **Ambas con índice primario: reunión por mezcla**. Están ordenadas físicamente por el atributo de reunión, una sola pasada.
- **Ambas con índice secundario: reunión por bloques**. No están ordenadas (el merge exigiría ordenar) y el indexado hace una búsqueda por tupla con un bloque por registro, que sale carísimo.

Ejercicio 3.3: ordenada pero sin índice (instructor y teaches)

instructor y teaches están ordenadas por legajo; teaches no tiene índice.

- Tamaño de la reunión: legajo es clave de instructor y foránea en teaches → 500 · 800 / 500 = **800 tuplas**.
- Algoritmo: **merge join**. Ambas ordenadas, sin ordenar nada, costo b_instructor + b_teaches. El indexado no se puede (no hay índice en teaches).
- La selección sobre la reunión usa **búsqueda lineal**: opera sobre un intermedio, que no tiene índices.

Qué recordar: "ordenada" y "con índice" son cosas distintas. Un índice primario necesita ambas: archivo ordenado y la estructura B+ encima.

Ejercicio 3.4: σ venta_id < x (ventas) ⋈ vendedores (recuperatorio 21/11/2023)

Índice primario en venta_id en ambas. Respuesta: **merge join**. Con A < v el índice primario no se usa: se escanea desde el principio hasta la primera tupla ⩾ x. El resultado es un prefijo de ventas, que sigue ordenado por venta_id, y vendedores también está ordenada. Costo b_σ + b_vendedores.

---

4. Costo total con materialización

Ejercicio 4.1: ventas (recuperatorio 21/11/2023, ej. 1b)

π cantidad, valor (σ fecha=x (ventas)). ventas tiene 1.000.000 registros en 600 bloques. 1/10 son del día x. Un registro con solo cantidad y valor ocupa la mitad.

Árbol: π sobre σ sobre ventas.

Operador	Entrada	Salida	Por qué
σ fecha=x	600	60	Sin índice sobre fecha: búsqueda lineal. Tuplas completas: 600/10
π cantidad, valor	60	30	Lee el intermedio. Registros de la mitad de tamaño

Costo total: 600 + 60 (escribir) + 60 (leer) = **720 bloques**. El resultado final de 30 bloques no se escribe.

Ejercicio 4.2: profe y materia (apunte, capítulo 5)

π legajo, nombre (σ nombre="Intro a los Algoritmos" (profe ⋈ materia)). profe: 2000 registros, 500 bloques. materia: 100 registros, 20 bloques. Un profe está a cargo de una sola materia.

- fs de reunión = 1/|profe| = 1/2000 → |r| = 2000 · 100 / 2000 = **100 tuplas**.
- Una tupla de la reunión es aproximadamente el doble de grande → 2 por bloque → **50 bloques**.
- fs de σ = 1/100 → **1 tupla, 1 bloque**.
- Costos: merge 500 + 20 = 520; selección lineal 50; proyección 1.
- **Total = (520 + 50 + 1) + (50 + 1) = 622**.

Ejercicio 4.3: bibliotecarios (pregunta de parcial con comentarios del profesor)

π DNI (σ antigüedad>5 (bibliotecario) ⋈ σ nombreBib='FaMaf' (trabajaEn)). bibliotecario: 3 bloques, 600 registros, índice secundario en antigüedad. trabajaEn: 32 bloques, 2048 registros, índice primario en nombreBib. 35 antigüedades, 200 bibliotecas, 1024 DNI en trabajaEn, M = 6.

- σ antigüedad > 5: fs = (35 − 6)/(35 − 1) = 29/34 → ≈ **512 tuplas** → ⌈512/200⌉ = **3 bloques**.
- σ nombreBib = 'FaMaf': fs = 1/200 → ≈ **10 tuplas** → **1 bloque**.
- **Reunión:** hay que usar los **intermedios**, no las tablas originales. V(DNI) en el primer intermedio = 512, en el segundo ⩽ 10 → fs = 1/512 → 512 · 10 / 512 = **10 tuplas**.
- Algoritmos: la primera selección usa **búsqueda lineal** (el secundario no conviene con 512 registros, casi toda la tabla); la segunda usa **índice primario, igualdad, clave no candidata** (h + b); la reunión usa **loop anidado por bloques con M buffers**.
- Costo de la reunión: ⌈1/(6−2)⌉ · 3 + 1 = **4 transferencias**.

Los tres errores que marcó el profesor: factor de reunión con tablas originales en vez de intermedias, usar el índice secundario en la primera selección, y olvidar calcular el costo de la reunión.

---

5. Índices posibles y costo de selección

Ejercicio 5.1: qué índices sirven (parcial 26/10/2023, ej. 1)

σ fecha=b ∧ x ⩽ venta_id ⩽ y (ventas).

- **Primario en fecha:** sirve (igualdad), costo h + b y después se filtra el rango.
- **Secundario en fecha:** sirve solo si hay **pocos registros por fecha**.
- **Primario en id:** sirve (rango), se busca x y se lee secuencial hasta y. Conviene si el rango es chico.
- **Secundario en id:** sirve solo si el rango trae **pocos registros**.

Se puede usar un índice y filtrar la otra condición sobre lo recuperado.

Ejercicio 5.2: índice secundario con 20 ids por bloque

σ venta_id = x, 1.000.000 de ventas, índice secundario, clave candidata.

- K = 1.000.000, base ⌈20/2⌉ = 10 → h = $\lceil \log_{10}(10^6) \rceil = 6$.
- Costo = h + 1 = **7** (con base 20 directa saldría 6; la opción 7 es la coherente con la fórmula n/2).

---

6. Orden de reuniones con programación dinámica

Regla: costo de una partición {S1, S2} = costo(S1) + costo(S2) + tamaño(S1) + tamaño(S2). El tamaño de una tabla base **no se suma** (ya existe). El de un par o trío intermedio **sí**. Costo 0 y tamaño 0 no son lo mismo.

Ejercicio 6.1: S, T, U (parcial 06/11/2025 y recuperatorio 18/11/2025)

S(b,c), T(c,d), U(d,a), 1000 tuplas cada una. V(S,c) = 500, V(T,c) = 20, V(T,d) = 50, V(U,d) = 1000. V(U,a) y V(S,b) no se usan (no son atributos de reunión).

Pares:

Par	Cuenta	Size
{S,T}	10^6 / max(500, 20)	2000
{T,U}	10^6 / max(50, 1000)	1000
{S,U}	sin atributo común: producto cartesiano	1.000.000

Todos con costo 0.

Trío {S,T,U}:

Partición	Plan	Costo
{{S,T},{U}}	(S ⋈ T) ⋈ U	2000
{{T,U},{S}}	(T ⋈ U) ⋈ S	**1000**
{{S,U},{T}}	(S ⋈ U) ⋈ T	1.000.000

- **Partición ganadora:** {{T,U},{S}}. **Mejor plan:** (T ⋈ U) ⋈ S, costo 1000.
- **Tamaño de (S ⋈ T) ⋈ U:** 2000 · 1000 / max(V(S⋈T, d) = 50, 1000) = **2000 tuplas**. El tamaño final no depende del orden.

Variante "profunda a la izquierda" (recuperatorio): para cada tabla r del conjunto X se calcula costo(X − {r}) + tamaño(X − {r}) y se elige el menor. La r es la tabla base que queda a la derecha. Respuesta corta:

> Para cada tabla r de X calculo costo(X−{r}) + tamaño(X−{r}) (r es tabla base y no se suma). Me quedo con la r de menor valor; el plan es (mejor plan de X−{r}) ⋈ r.

Con 3 tablas el resultado es idéntico al general, porque toda partición es 2+1. Recién con 4 o más tablas el general evalúa también las particiones 2+2.

Ejercicio 6.2: cliente, pizzas, pedido (parcial 2023, ej. 1b)

cliente 500, pizzas 40, pedido 5000. Pares: {cliente,pizzas} = 20.000, {cliente,pedido} = 5000, {pizzas,pedido} = 5000, todos con costo 0.

Partición	Costo
{{cliente,pizzas},{pedido}}	20.000
{{cliente,pedido},{pizzas}}	**5000**
{{pizzas,pedido},{cliente}}	**5000**

- Tamaño: 5000 · 40 / max(40, 40) = **5000**.
- Respuesta: tamaño 5000, costo 5000, mejor plan (pedido ⋈ cliente) ⋈ pizzas. Hay empate con (pizzas ⋈ pedido) ⋈ cliente; lo importante es descartar la de 20.000.

Ejercicio 6.3: para practicar (con solución)

A(x,y) 100 tuplas, B(y,z) 200, C(z,w) 50. V(A,y) = 20, V(B,y) = 50, V(B,z) = 10, V(C,z) = 10.

- Pares: {A,B} = 100·200/50 = 400. {B,C} = 200·50/10 = 1000. {A,C} = 5000 (producto cartesiano).
- Trío: ganadora {{A,B},{C}}, plan (A ⋈ B) ⋈ C, costo 400.
- Tamaño: 400 · 50 / 10 = **2000**.

---

7. Optimización heurística

Reglas: selección temprana, proyección temprana, selección más restrictiva primero, reunión profunda a la izquierda, combinar selecciones con producto cartesiano para formar reuniones.

Ejercicio 7.1: curso, profe, teléfono

π legajo, numero (σ nombre="optimizador de consultas" (curso ⋈ (profe ⋈ telefono))). nombre está solo en curso.

1. **Selección temprana y más restrictiva:** π legajo, numero ((σ nombre="..."(curso)) ⋈ (profe ⋈ telefono)).
2. **Proyección temprana:** de curso solo legajo, de profe solo legajo, de telefono legajo y numero.
3. **Profunda a la izquierda:** ((π legajo σ nombre="..."(curso) ⋈ π legajo(profe)) ⋈ π legajo,numero(telefono)).
4. **Producto cartesiano:** no hay, no aplica.

Final: π legajo, numero ((π legajo(σ nombre="..."(curso)) ⋈ π legajo(profe)) ⋈ π legajo,numero(telefono)).

Ejercicio 7.2: pizzería con heurística y luego programación dinámica

π cnombre, cid, zid (σ znombre='muzarella' ∧ paño>2015 (cliente ⋈ pizzas ⋈ pedido)). Primero heurística, después programación dinámica.

- **Heurística:** bajar znombre a pizzas (la más restrictiva, 4 tuplas de 40), paño a pedido, y proyectar: pizzas′ (zid), pedido′ (cid, zid), cliente′ (cid, cnombre).

- **Programación dinámica** con los tamaños ya filtrados (supuse paño > 2015 ≈ 4500 tuplas **[supuesto]**): {cliente, pizzas′} = 2000 (producto cartesiano), {cliente, pedido′} = 4500, {pizzas′, pedido′} = 4·4500/40 = **450**.
- **Resultado:** π cnombre, cid, zid ((π zid σ znombre='muzarella′(pizzas) ⋈ π cid,zid σ paño>2015(pedido)) ⋈ π cid,cnombre(cliente)).

---

8. Recuperación de información: respuestas modelo

8.1 Tuplas de la lista invertida (se repite)

- **Modelo vectorial:** (id_documento, peso), con el peso dado por la frecuencia o por TF-IDF.
- **Con proximidad:** (id_documento, posición).
- **Para SAME:** (id_documento, nro_párrafo, posición), ordenadas por documento, párrafo y posición.

8.2 Procesar una consulta (se repite)

Los 3 pasos del apunte, con coseno si es vectorial:

1. Buscar cada término de la consulta en el **vocabulario** (ordenado lexicográficamente), después de sacar stopwords y aplicar stemming.
2. Recuperar las **listas invertidas** de esos términos.
3. Aplicar la lógica booleana con conjuntos: AND es intersección, OR es unión, NOT es resta del total.
4. (Solo si es por relevancia) calcular la **similitud del coseno** entre el vector de la consulta y el de cada candidato, y ordenar de mayor a menor.

8.3 Construir un índice invertido (modelo booleano)

1. Tokenizar cada documento (separar palabras, sacar puntuación, minúsculas).
2. Eliminar stopwords.
3. Aplicar stemming (Porter).
4. Armar el vocabulario ordenado lexicográficamente.
5. Representar cada documento como conjunto de términos.
6. Invertir: pares (término, id_documento) ordenados, sin duplicados.
7. Guardar cada lista en bloques consecutivos, con un B+ que mapea término a lista.

En booleano no hacen falta frecuencias ni pesos.

8.4 Estructuras almacenadas que usa una consulta (solo nombres)

Vocabulario, índice invertido con la lista invertida de cada término, árbol B+ que mapea término a lista. Opcionalmente, los vectores de los documentos.

8.5 Vocabulario según el tipo de colección (se repite)

Noticias web o relatos de fútbol: **entidades nombradas** (personas, organizaciones, lugares, fechas, eventos, montos), más frases o n-gramas y sinónimos agrupados por concepto. Para fútbol: jugadores, técnicos, árbitros, equipos, estadios, torneos, eventos del juego (gol, penal, tarjeta roja, offside), minutos y resultados.

8.6 XOR con conjuntos

Si S1 y S2 son los conjuntos de documentos de T1 y T2: **T1 XOR T2 = (S1 ∪ S2) − (S1 ∩ S2)**, equivalente a (S1 − S2) ∪ (S2 − S1).

8.7 Operador SAME

L(t) = tuplas de la lista invertida de t:

t1 SAME t2 = { d | ∃p ∃i ∃j : (d, p, i) ∈ L(t1) ∧ (d, p, j) ∈ L(t2) }

Es una intersección que compara el par (documento, párrafo), no solo el documento.

8.8 Componentes de un SRI web que no tiene uno de archivos de PC

Crawler (recorre y descarga páginas siguiendo enlaces) y análisis de enlaces o ranking por popularidad (tipo PageRank). **No salen de tu teórico**; confirmalo con tus apuntes de clase.

8.9 Tipos de contexto y prompt (2024)

No vienen del teórico. Propuesta para el texto de los incendios: contexto **geográfico** ("Sierras de Córdoba"), **temporal** ("incendios históricos", "frecuencia creciente"), **cultural** ("tradición oral y cultural de la región"). Para el prompt, incluí los cuatro modificadores (rol, audiencia, tipología de texto, estructura) en 5 líneas o menos y etiquetalos. Revisalo con tus apuntes.

---

9. Errores típicos para revisar antes del examen

1. Calcular los bloques de la selección con la tupla ya proyectada: la selección deja tuplas completas.
2. Olvidar que el intermedio se **escribe una vez y se lee otra**.
3. Usar como K de la altura del B+ el total de registros: es la cantidad de valores **distintos** de la clave.
4. Usar la tabla grande como externa en el loop anidado.
5. Suponer que el índice siempre conviene: comparar $n_r \cdot c$ con $b_r \cdot b_s$.
6. Sumar b_f dos veces.
7. En merge, resta o intersección, olvidar el costo de **ordenar** si no vienen ordenadas.
8. Calcular factores de reunión con las tablas originales cuando hay selecciones previas: usar los **intermedios**.
9. En programación dinámica, sumar el tamaño de una tabla base, o confundir costo 0 con tamaño 0.
10. Olvidar anotar los pares sin atributo común (producto cartesiano).
11. Usar un índice secundario para rangos que traen muchos registros.

---

10. Qué no cubrimos

- **Dependencias funcionales y normalización (FNBC):** aparecen en el recuperatorio del 21/11/2023 (ejercicios 4 y 5) y no los resolvimos.
- La primera hoja del parcial del 06/11/2025 estaba ilegible, así que su ejercicio 1 se reconstruyó a partir de la foto que pasaste después.
- El recuperatorio del 18/11/2025 tiene un Ejercicio 2 cuyo enunciado se cortó: lo resolvimos con la consigna de la foto.